

University of Dhaka (DU)

Department of Computer Science & Engineering



DubsiBhai

Drainage Waterlogging Management for Dhaka

Functional and Non-functional Requirements Report

With High-Level System Architecture

Course: CSE 4113 — Internet Programming Lab

Team: AI_TookMyJob

Team Members

Mehedi Hasan	Student ID: 22
Mahmudur Rahman Moin	Student ID: 30
Nafisha Akter	Student ID: 40
Biplob Paul	Student ID: 50

October 2026

Requirements baseline v1.0

Contents

1	Introduction and Scope	2
1.1	Purpose	2
1.2	Problem and project objective	2
1.3	Scope of the semester project	2
1.4	Existing prototype versus required final system	2
1.5	Exclusions and assumptions	2
2	Stakeholders and Business Rules	3
2.1	Actors and access scope	3
2.2	Complaint lifecycle	3
2.3	Publication, priority, and terminology	3
3	Functional Requirements	4
3.1	Accounts and Access Control	4
3.2	Complaint Submission and Evidence	5
3.3	Moderation, Assignment, and Resolution	6
3.4	Public Tracking, Notifications, and Analytics	7
3.5	Machine Learning and Geographic Insights	8
4	Non-functional Requirements	9
4.1	Security and Privacy	9
4.2	Performance, Usability, and Compatibility	10
4.3	Reliability, Operations, and ML Quality	11
5	High-Level System Architecture	12
5.1	Architectural decision	12
5.2	Component responsibilities and data ownership	13
5.3	Complaint-to-classification flow	13
5.4	Development and deployment boundaries	13
6	Verification and Traceability Plan	14
6.1	Verification methods	14
6.2	Survey needs to acceptance scenarios	14
6.3	Test data and evidence record	14
7	Delivery and Acceptance Baseline	15
7.1	Implementation sequence	15
7.2	Definition of acceptance	15
7.3	Change control and unresolved dependencies	15
7.4	Project sources	15

How to use this report

Each FR or NFR identifier denotes a testable obligation for the final semester implementation. Acceptance criteria define the evidence needed to mark it passed. These are proposed project targets, not claims that all features or quality targets have already been achieved.

1 Introduction and Scope

1.1 Purpose

This report specifies the functional requirements, non-functional requirements, and high-level architecture of **DubsiBhai**. It converts the selected project idea and earlier stakeholder survey into obligations that can be checked after implementation. It is intended for the course instructor, development team, testers, and demonstration reviewers.

1.2 Problem and project objective

Residents need a structured way to report drainage problems and follow their resolution. Municipal workers need complaint details, locations, and clear assignments. Administrators need tools to verify reports, manage work, and identify recurring problem areas. DubsiBhai shall connect these activities through a bilingual web application with traceable complaint records and Bengali text classification.

1.3 Scope of the semester project

The requirements baseline covers account access; geolocated complaints and photos; manual moderation; ward-scoped assignments; resolution evidence; public status and map views; notifications; operational analytics; Bengali classification; human-reviewed duplicate suggestions; and complaint-based hotspot summaries. The baseline contains **28 functional requirements and 15 non-functional requirements**. Implementation may be phased, but phases do not silently remove requirements from the final acceptance scope.

The system supports residents, municipal workers, ward administrators, super administrators, and anonymous visitors. It records reported and verified incidents; it does not certify that a road is safe to travel or that a municipal response will occur within a guaranteed time.

1.4 Existing prototype versus required final system

At the preparation of this report, the repository contains email/password and Google sign-in, revocable sessions, email verification/reset, a standalone Bengali classifier, a base-map preview, and a Docker development stack. Complaint workflows, staff dashboards, live complaint overlays, durable ML jobs, duplicate detection, and clustering remain planned. Existing functions still require acceptance evidence; presence of code alone is not a pass.

Design refinements since the survey report

The implementation uses revocable HttpOnly session cookies rather than the earlier JWT sketch. The API-owned background worker will invoke the ML service over HTTP and persist results; the ML service will not independently write complaint records. ML suggests category, while an administrator controls severity and final decisions.

1.5 Exclusions and assumptions

Native mobile apps, IoT water-level sensors, SMS/phone verification, automated worker routing, weather forecasting, comments/upvotes, and multi-city expansion are future scope. Google sign-in requires an externally configured OAuth client; map tiles require network access. Ward boundaries, staff accounts, and evaluation datasets must be supplied or clearly labelled as demonstration fixtures.

2 Stakeholders and Business Rules

2.1 Actors and access scope

Actor	Main activities	Permitted private data
Public visitor	Browse published reports, map and hotspots.	None; public projections only.
Resident	Submit, edit eligible own reports, track progress, request reopening.	Own full complaints and account.
Municipal worker	View assignments, start work, submit resolution evidence.	Current assignments and permitted work history.
Ward administrator	Verify/reject, publish, assign, review ML suggestions and analytics.	Complaints and staff in assigned ward.
Super administrator	Provision staff, manage roles/wards, view city analytics and ML operations.	All wards; privileged actions audited.

Public registration creates residents only. Staff accounts are provisioned by a super administrator. One account has one role in the initial design; staff roles do not automatically inherit resident-only submission rights. A worker or ward administrator must have an assigned ward. An anonymous visitor has no account role.

2.2 Complaint lifecycle

Transition	Authorized actor	Required condition
Reported → Verified	Scoped administrator	Review and moderation completed.
Reported → Rejected	Scoped administrator	Recorded rejection reason.
Verified → Assigned	Scoped administrator	Eligible active worker in same ward.
Assigned → In progress	Assigned worker	Current active assignment matches actor.
In progress → Resolved	Assigned worker	Resolution note and evidence photo.
Resolved → Reported	Scoped administrator	Approved reopening request with reason.

An in-progress reassignment closes the previous assignment and returns the complaint to Assigned for the new worker. Illegal transitions must fail without partial updates. Each effective transition creates a timestamped history entry.

2.3 Publication, priority, and terminology

Workflow state and publication are separate. New reports and rejected complaints remain private. Verification permits publication only after review of summary, coordinates, and photos. Public map locations are rounded to a 0.001-degree grid (approximately 100 metres in Dhaka); exact locations remain available to authorized users. Administrators select severity from Low, Medium, High, and Critical.

Category describes the type of drainage problem; **severity** is an administrator’s urgency decision; **status** records workflow progress. “Live updates” mean refreshable complaint status within the stated update window, not continuous sensor measurements. A hotspot is a pattern in recorded complaints, not a flood forecast.

3 Functional Requirements

Each requirement shall be verified with valid, invalid, and unauthorized cases where applicable. The acceptance criteria specify observable outcomes; the implementation may choose interfaces that preserve these outcomes.

3.1 Accounts and Access Control

ID	Requirement (shall)	Acceptance criterion / evidence
FR-01	Provide resident registration using name, email, and password; never accept a privileged role from public registration.	A valid unique registration creates a resident. Invalid input is rejected. Supplying worker/admin role cannot grant it. Repeated email registration creates no duplicate account and reveals no account-existence detail.
FR-02	Authenticate by email/password and Google identity, establish a revocable session, and support sign-out from one or all sessions.	Both methods return the correct profile. Invalid credentials fail; replayed Google nonce fails. Sign-out invalidates the corresponding session(s). Existing email accounts are not automatically linked to Google by matching email.
FR-03	Provide email verification and password reset using expiring, single-use links. Require a verified email for complaint submission.	A valid link performs its action once; reused/expired links fail. Reset revokes existing sessions. An unverified resident cannot submit; a verified resident can. Email delivery is visible in the development inbox.
FR-04	Show the signed-in user's profile and active sessions, and allow revocation of an individually selected own session.	The profile identifies the actual session user. Revoking a selected session prevents its next authenticated action. A user's request cannot list or revoke another user's sessions.
FR-05	Authorize every private read and mutation by role, ward, ownership/assignment, activity, and applicable complaint state.	Test all five actor types. Another resident's private record is unavailable; a worker cannot update another assignment; a ward admin cannot access another ward. Disabled accounts and revoked sessions cannot act.
FR-06	Allow super administrators to provision staff, assign/change roles and wards, and deactivate accounts with an audit record.	A super admin creates a scoped worker and ward admin. A ward admin/resident cannot grant roles. A deactivated user's next protected request fails; every effective privileged change records actor, target, and timestamp.

Session design

The final system shall retain the current server-side session approach. Session tokens and Google credentials shall not be stored in browser local storage. Account provisioning is an administrative workflow, not a public role selector.

3.2 Complaint Submission and Evidence

ID	Requirement (shall)	Acceptance criterion / evidence
FR-07	Allow a verified resident to submit a drainage complaint containing description, confirmed coordinates, and optional initial photos.	A valid submission returns a unique complaint ID and Reported status. Reporter identity comes from the session. The persisted report can be retrieved by its owner after API/container restart. Anonymous and staff submission attempts fail.
FR-08	Validate description length from 1 to 5000 characters after trimming, coordinates, and service-area membership; derive ward from configured boundaries.	Blank/5001-character text, malformed coordinates, or a point outside supported wards is rejected. A valid boundary fixture yields its expected ward. A submitted ward value cannot override the server's determination.
FR-09	Offer user-triggered geolocation and manual map-pin placement; require confirmation of the selected location before submission.	Allowing geolocation fills a reviewable pin. Denying permission still permits manual selection. No location is silently submitted; the confirmed coordinates match those saved in the private record.
FR-10	Support up to five complaint/evidence photos, each at most 5 MiB, with decoded JPEG, PNG, or WebP content. Strip image metadata and use private storage.	Boundary-size valid images succeed. Excess count/size, malformed images, and a renamed non-image fail. Stored images contain no original EXIF location metadata; unapproved photos are not publicly accessible.
FR-11	Allow the owning resident to edit description/location only while Reported; increment the revision and schedule classification of updated text.	An eligible edit persists and increments revision. Other actors or later-state edits fail. A classification result for the previous revision cannot replace the current result. Historical workflow entries remain intact.
FR-12	Provide server-scoped, paginated complaint lists and private detail with evidence, classification state, and chronological status history.	An owner sees only eligible own private reports; worker/admin scopes follow FR-05. Page size defaults to 20 and cannot exceed 100. Filtering and pagination produce no missing/duplicated records on an unchanged fixture.

Submission and upload separation

The API creates the complaint independently of photo upload. A failed upload must not create a second complaint or remove the first record; the UI retains the complaint ID and permits retry of the failed attachment.

3.3 Moderation, Assignment, and Resolution

ID	Requirement (shall)	Acceptance criterion / evidence
FR-13	Allow a scoped administrator to verify or reject a Reported complaint, record a reason for rejection, and set severity independently of ML category.	Same-ward review succeeds; cross-ward review fails. Rejection without a reason fails. Allowed severity values persist. Classification cannot verify/reject a report or silently change its administrator-selected severity.
FR-14	Provide publication controls for a moderated public summary, approximate location, and individually approved photos; allow hiding a published record.	Unreviewed reports remain private. Publishing exposes only approved fields/photos. Hiding removes the report from the next public detail/list/map response; private owner/staff access remains available.
FR-15	Allow administrators to assign verified complaints to eligible active workers in the same ward, with at most one active assignment per complaint.	A valid assignment sets Assigned status. Wrong-ward/inactive workers fail. Two concurrent assignment requests result in one active assignment; the losing conflicting request returns a safe conflict response.
FR-16	Show each worker their current assignment queue, complaint details, precise work location, severity, and status history.	Seed assignments for two workers. Each worker's queue contains only their permitted records. Sorting by severity then assignment time is deterministic. Detail includes the work pin and approved private evidence.
FR-17	Allow the assigned worker to start work and resolve it with a nonblank resolution note and at least one evidence photo; close the active assignment on resolution.	Assigned-to-In-progress and In-progress-to-Resolved succeed for the assignee. Missing evidence/note, a different worker, or an illegal transition fails. Resolution, assignment closure, and history commit together.
FR-18	Support audited reassignment and resident reopening requests; only a scoped administrator may approve reopening a Resolved complaint.	Reassignment closes the old assignment and creates one new active assignment atomically. A resident request alone changes no status. Approval returns the report to Reported, preserves past evidence/history, and records the reason.

Concurrency rule

Status and assignment writes shall use a revision check or equivalent transaction locking. A rejected or stale update must leave both the complaint state and assignment history unchanged.

3.4 Public Tracking, Notifications, and Analytics

ID	Requirement (shall)	Acceptance criterion / evidence
FR-19	Provide anonymous browsing of published complaint summaries and a moderated public timeline, with ward, status, severity, and category filters.	A known fixture produces the expected filtered IDs. Unpublished/rejected/hidden IDs return unavailable. Public responses omit reporter/contact information, exact coordinates, internal notes, raw private text, and object-storage keys.
FR-20	Display published complaints on an interactive map with viewport queries and an equivalent list view.	Moving the viewport updates eligible markers; filters match the list. All markers use the public rounded coordinates. A fixture exceeding the feature cap returns aggregated/capped results rather than an unbounded response.
FR-21	Refresh tracking and public complaint views so committed changes are reflected within 30 seconds while connected; provide clear empty, loading, and error states.	After an authorized update, a continuously open eligible view displays the new state within 30 seconds. Empty fixtures show no invented reports. Network failure displays an error/retry state without falsely indicating successful refresh.
FR-22	Create in-app notifications for the reporter when a complaint is verified, rejected, assigned, resolved, or reopened, with read/unread tracking.	Each effective transition creates one notification per recipient/event. Retry creates no duplicate effective notification. Only the recipient reads or marks it read. Notification appears within 30 seconds under the test workload.
FR-23	Provide ward and city analytics for complaint counts, backlog, resolution rate, and mean resolution duration over a selected date range.	A hand-calculated fixture matches each metric. A ward admin cannot query another ward; super admin can query city totals. Empty denominators show N/A, not division errors or misleading zero-duration results.
FR-24	Present resident and worker flows in Bengali and English, retain language choice, and show equivalent field validation and status labels in both languages.	Switch language on login, submission, tracking, map/list, and worker pages. Controls and messages retain meaning; Bengali text persists correctly. Reload retains the choice. Missing translation does not produce an empty control.

Analytics definitions. Backlog counts nonterminal, nonrejected complaints at the query cutoff. Resolution rate is the number currently Resolved divided by all nonrejected complaints created in the selected interval. Mean resolution duration uses creation-to-first-resolution time for complaints first resolved within that interval. Reopened records retain their first-resolution timestamp. Dates are stored in UTC and displayed in Asia/Dhaka time.

3.5 Machine Learning and Geographic Insights

ID	Requirement (shall)	Acceptance criterion / evidence
FR-25	Classify Bengali complaint descriptions asynchronously into the six supported categories and store suggestion, probabilities, revision, model version, and processing state separately from human decisions.	A committed complaint is accepted without waiting for inference. A processed job stores valid metadata for its current revision. Replaying a job or delivering a stale result has no extra effect. Administrator category overrides remain unchanged.
FR-26	Suggest possible duplicates using text similarity plus geographic and time constraints; require a scoped administrator to confirm a duplicate relationship.	Use a labelled fixture with qualifying and nonqualifying pairs. Suggestions respect the configured similarity threshold, 100-metre radius, and 14-day window. Suggestion alone merges/deletes nothing. A confirmed relationship is audited and a report cannot duplicate itself.
FR-27	Run an administrator-triggered geographic clustering job and publish complaint-based hotspot summaries with a run timestamp and configurable recurrence labels.	A seeded geographic/time fixture produces expected memberships under documented parameters. Incomplete runs are not published. Public output is aggregated and privacy-filtered. Changing the observation window changes recurrence results as defined below.
FR-28	Expose job status to scoped administrators, allow retry of failed current-revision ML jobs, and provide explicit versioned retraining and model readiness checks.	An unavailable model returns not-ready rather than a successful prediction. Retry coalesces an already-active job. A validated retrained artifact loads after restart; replacing it with the previous trusted artifact supports rollback.

Classification categories: blocked drain, open manhole, road flooding, recurring waterlogging, sewage overflow, and other. Confidence is a model probability, not a correctness guarantee or verified severity estimate.

Initial duplicate acceptance configuration: cosine similarity at least 0.80 using a documented, versioned embedding model; geographic distance at most 100 metres; creation-time difference at most 14 days. Test each boundary and exclude the complaint itself. These thresholds are configurable review heuristics, not a guarantee that two reports describe the same incident.

Initial clustering acceptance configuration: use verified, nonduplicate, nonhidden complaint locations from a 90-day observation window; DBSCAN geographic radius 100 metres, minimum three distinct complaints. Evaluate distance in metres with an appropriate geographic calculation. Label a cluster Occasional when it has reports on one distinct calendar date, Recurring on two to four dates, and Chronic on five or more dates. These are project-defined complaint recurrence labels, not scientific flood-risk categories. Report the count and window with each label. Round public centroids to the same 0.001-degree grid as public pins.

Human control and reproducibility

The chosen algorithms and thresholds shall be documented with the test fixtures. ML must not autonomously verify, reject, assign, resolve, or merge complaints. Duplicate suggestions and geographic recurrence need different evidence from a text-category training CSV.

4 Non-functional Requirements

The following targets apply to the final system. Where deployment security is specified, the HTTP-only localhost development stack is an explicit exception. Measurements must state their environment, dataset, duration, and result.

4.1 Security and Privacy

ID	Requirement (shall)	Acceptance criterion / evidence
NFR-01	Protect passwords with Argon2id hashing and browser sessions with HttpOnly cookies; use Secure cookies and HTTPS in a deployed environment.	Inspect a stored password hash and session cookie flags. Plain passwords/tokens never appear in API responses, logs, or browser storage. An expired, revoked, or disabled-account session fails on its next protected request.
NFR-02	Validate allowed Origin and session-bound CSRF tokens for unsafe browser requests; enforce exact credentialed CORS origins and authentication rate limits.	Missing/wrong CSRF and untrusted origins cannot mutate data. A foreign origin cannot read credentialed responses. Exceeding the configured auth rate limit yields 429. Redis failure rejects authentication safely with 503.
NFR-03	Enforce object-level access and public-data minimization consistently across detail, list, map, photo, analytics, and notification routes.	A permission matrix covers every role and at least two wards. All negative cases are denied. Automated checks find no contact data, exact coordinates, raw private text, or internal storage keys in public responses.
NFR-04	Keep secrets out of source control, frontend bundles, image layers, and routine logs; restrict deployed ML, databases, and private photo storage to authorized/internal access.	Inspect build context exclusions and built frontend assets. Only the public OAuth client ID is exposed. Unauthenticated direct deployed inference/storage requests fail; databases are not publicly reachable. Errors disclose no credentials or stack traces.
NFR-05	Maintain a durable, append-only application audit trail for privileged account changes, moderation, publication, assignment, and workflow transitions.	Each successful action has actor, target, old/new value or action, UTC timestamp, and request ID. A normal user cannot edit/delete audit entries. Failed transactions leave no misleading successful-change entry.

Security boundaries

Authorization is enforced by the API, not by hidden frontend controls. Reporter identity and ward scope come from trusted server records. A public complaint response is a separate projection rather than a private record with selected UI fields hidden.

4.2 Performance, Usability, and Compatibility

ID	Requirement (shall)	Acceptance criterion / evidence
NFR-06	Under the defined test workload, provide p95 read latency at most 1 second and complaint creation latency at most 2 seconds, excluding separate photo upload.	After a 60-second warm-up, run 20 concurrent clients for 10 minutes against 10000 seeded complaints. Measure public/private list, detail, and map reads separately and complaint creation separately. Each p95 meets its target; unexpected 5xx rate is below 1%.
NFR-07	Bound response sizes and asynchronous work; return at most 100 list records and at most 1000 map features per request, and complete p95 classification within 60 seconds when healthy.	Maximum-size and dense-viewport fixtures respect limits or return documented aggregation. Submit 100 jobs under the test load and measure commit-to-result time; p95 is at most 60 seconds. Photo storage/inference are not serialized into complaint creation.
NFR-08	Support resident and worker flows on mobile and desktop, including keyboard operation, visible focus, labelled controls, and text-readable validation/status.	Check widths 360, 768, and 1366 pixels without horizontal page overflow. Complete core flows using keyboard only. Ordinary text contrast is at least 4.5:1; status is not conveyed by colour alone. Denied geolocation remains usable.
NFR-09	Allow representative residents to complete complaint submission with clear confirmation and recovery feedback.	Five participants unfamiliar with the form attempt a valid report after sign-in. At least four complete it within 3 minutes without facilitator intervention, excluding authentication/network outages. Record tasks, times, failures, and improvements; do not invent participant results.
NFR-10	Support the current stable Chrome, Edge, and Firefox releases available at final evaluation; preserve UTF-8 Bengali data and language preferences.	Record tested browser versions. Login, submission, tracking, map/list, and worker updates pass the same checklist in each browser. A Bengali description round-trips exactly, and language preference survives refresh in each.

Benchmark environment. Use the containerized stack on a documented machine with at least four logical CPU cores, 8 GB RAM, and SSD storage, with at least 4 GB assigned to Docker. Run load generation from a separate process, use a warmed application build, and exclude external Google authentication, map-tile downloads, and SMTP delivery from API timing. Results from a different machine must state its specifications. Targets are proposed acceptance values, not measured prototype results.

4.3 Reliability, Operations, and ML Quality

ID	Requirement (shall)	Acceptance criterion / evidence
NFR-11	Keep submitted complaints durable when ML or Redis processing is unavailable, with transactional outbox recovery, idempotent jobs, and protection against stale results.	With a valid existing session, disable inference/queue publication and submit a report. The complaint and pending durable job commit. Restore services: the job completes within 5 minutes under light load. Replayed/stale jobs cause no duplicate/current-result overwrite.
NFR-12	Achieve at least 99% successful eligible application requests in a two-hour controlled healthy-service soak test, excluding intentional denial cases and third-party outages.	Record request totals, failures, and availability as successful eligible requests divided by attempted eligible requests. No committed data is lost. Authentication failure during an intentional Redis outage is recorded separately, not concealed.
NFR-13	Preserve data across container recreation and support a documented daily backup/restore procedure with a 24-hour maximum backup age and 30-minute recovery target.	Create known database and photo records, recreate containers without deleting volumes, and compare values. Restore a backup and linked photos into an isolated environment within 30 minutes; verify relationships and counts. Redis alone is not the durable job source.
NFR-14	Provide reproducible container startup, migrations, health/readiness checks, structured safe diagnostics, and automated critical-path tests.	On a clean checkout with Docker Desktop and PowerShell, start.ps1 builds and starts all required services without host Node/Python installs. Record download time separately; after images are available, healthy startup takes at most 5 minutes. Authorization, transitions, and job recovery tests pass.
NFR-15	Evaluate classification reproducibly and report model/dataset provenance and per-class results without presenting synthetic scores as real-world accuracy.	Normalize/deduplicate before an 80/20 stratified split with seed 42. On the bundled synthetic fixture, macro F1 is at least 0.80. Save dataset hash, source, model version, accuracy, macro F1, confusion matrix, and per-class metrics. Real-data performance requires a separately labelled real evaluation set.

Availability and authentication distinction

NFR-11 concerns durable complaint processing during inference/queue failure for an already valid session. Authentication rate limiting remains fail-closed during Redis failure (NFR-02), so new logins may be temporarily unavailable. The current prototype's auth router behavior does not itself prove future complaint-outage recovery.

5 High-Level System Architecture

5.1 Architectural decision

DubsiBhai uses a **modular monolith** for transactional business logic and a **separate FastAPI ML service** for inference and model lifecycle. This keeps authorization, complaints, assignments, and history within one coherent API while allowing model dependencies and retraining to evolve independently. The diagram shows the *target architecture*; it is not a claim that all depicted modules are deployed today.

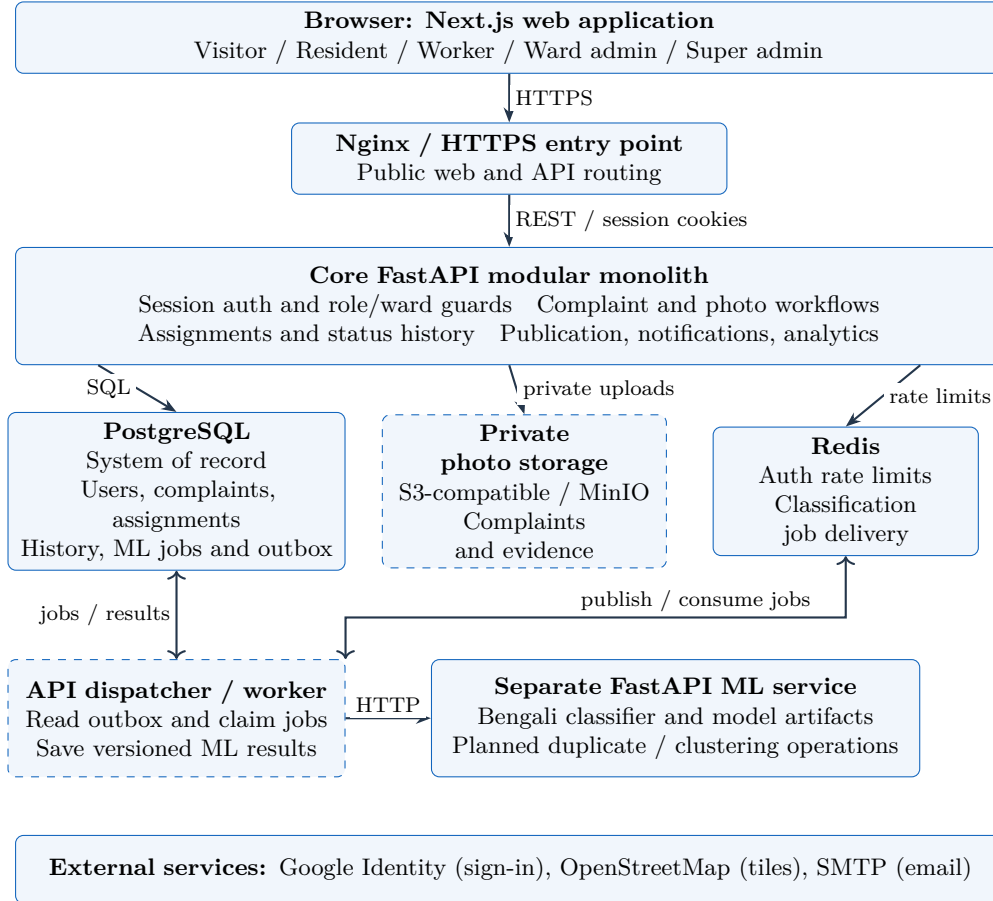


Figure 1: Target high-level architecture. Dashed boxes denote components not yet implemented in the prototype. Existing boxes may contain planned capabilities.

MongoDB is provisioned for development experimentation but has no required business-data ownership in this baseline. PostGIS/pgvector support and production HTTPS are planned; the current PostgreSQL image does not include those extensions.

5.2 Component responsibilities and data ownership

Component	Responsibility	Ownership / boundary
Next.js frontend	Bilingual forms, dashboards, public map/list, tracking.	Browser accesses business data through core API; UI checks do not authorize requests.
FastAPI core API	Sessions, authorization, transactional complaint lifecycle, evidence references, public projections.	Owns PostgreSQL schema, migrations, jobs, and final workflow decisions.
API background workers	Dispatch committed outbox events, consume jobs, call inference, save results and notifications.	Check job ID/revision; no duplicate effective writes; use API database layer.
ML FastAPI service	Category inference, versioned artifact loading, future duplicate/clustering calculations.	No direct complaint-table writes; no human account authority.
PostgreSQL	Accounts, sessions, complaints, assignments, timelines, results, outbox and audits.	Authoritative durable data. PostGIS/pgvector may support future spatial/vector queries.
Redis	Auth rate-limit counters and asynchronous job transport.	Recoverable transport/state; never sole storage for accepted complaints/jobs.
Object storage	Validated private photos and resolution evidence.	API controls approval and authorized access; references stored in PostgreSQL.

5.3 Complaint-to-classification flow

1. The resident confirms the report and submits it to the core API.
2. In one transaction, the API saves the complaint, current-revision ML job, and outbox event; it commits and returns the tracking ID.
3. The dispatcher publishes the committed job ID to Redis and retries failures.
4. The API-owned worker claims the job, loads current text, and invokes `/classify` on the private ML service.
5. The worker validates the response and revision, then stores the suggestion, model version, and job completion. Administrative decisions stay independent.
6. The UI refreshes the scoped record; failed jobs remain visible and retryable.

5.4 Development and deployment boundaries

The current local Compose stack has eight services: web, API, ML, PostgreSQL, Redis, MongoDB, Mailpit, and Nginx. Host access binds to loopback; applications inside Docker use service DNS names. `start.ps1` runs builds, migrations, initial synthetic training, and startup health checks with persistent data volumes. Mailpit is a development inbox. Photo storage and API workers will be added when their features are implemented. Production requires HTTPS, private data/ML services, deployment secrets, backups, and a real SMTP provider; the development Compose stack does not establish compliance with production requirements.

6 Verification and Traceability Plan

6.1 Verification methods

Functional tests demonstrate expected outcomes and denied operations. **Integration tests** exercise API, database, Redis, storage, and ML interactions. **Inspection** checks configurations, schemas, secret exposure, and audit fields. **Measurement** evaluates latency, availability, usability, and ML metrics. Each requirement must retain its own pass/fail evidence; a successful demonstration of one happy path does not certify access control or outage recovery.

6.2 Survey needs to acceptance scenarios

The survey report supplies stakeholder priorities; percentages are not repeated here as independently validated measurements. The following scenarios connect those needs to the requirement IDs and final evaluation evidence.

Test	Need / scenario	Requirements	Evidence
AT-01	Account entry and recovery	FR-01-04; NFR-01-02	Registration, two login methods, reset/replay and session-revocation results.
AT-02	Correct staff and ward access	FR-05-06; NFR-03-05	Role/ward permission matrix and privileged audit samples.
AT-03	Resident reporting with location/photos	FR-07-12; NFR-08-10	Submission/edit validation, denied location fallback, attachment/privacy checks.
AT-04	Verified, prioritized worker handling	FR-13-18; NFR-05	Legal/illegal transitions, competing assignments, evidence, reopening history.
AT-05	Visible progress and bilingual map	FR-19-24; NFR-03,08-10	Public projection snapshots, 30-second refresh timing, notifications, analytics fixture.
AT-06	Bengali categorization	FR-25,28; NFR-07,15	Versioned predictions, known-class validation, holdout report, retraining/rollback.
AT-07	Duplicate and recurring area review	FR-26-27; NFR-03	Spatial/time fixtures, no automatic merge, labelled complete hotspot runs.
AT-08	Acceptable response and stability	NFR-06-07,12	Per-endpoint p95 load report and two-hour soak counters.
AT-09	Outage and data recovery	FR-25,28; NFR-11,13	ML/queue outage recovery, replay/stale tests, isolated backup/restore comparison.
AT-10	Easy team setup and safe operation	NFR-04,14	Clean-checkout launch log, health checks, image/secret inspection and automated test results.

6.3 Test data and evidence record

Maintain at least two wards, two residents, two workers, scoped administrators, and a super administrator. Seed every workflow state, unpublished/hidden records, boundary locations, duplicate/nonduplicate pairs, and dated geographic clusters. The performance fixture contains 10000 complaints. Use synthetic/nonpersonal test data and an isolated environment for recovery and destructive tests.

For each ID, record build/commit, preconditions, fixture version, steps, expected result, observed result, evidence path, evaluator, date, and Pass/Fail/Not tested. Record known defects rather than labelling untested requirements as satisfied.

7 Delivery and Acceptance Baseline

7.1 Implementation sequence

1. **Foundation:** confirm account/session acceptance, role/ward scope, database migrations, and reproducible Docker setup.
2. **Core workflow:** private submission, evidence storage, moderation, assignment, status history, and reopening.
3. **Visibility:** privacy-filtered map/list, notifications, bilingual flows, and fixture-verified analytics.
4. **ML and insight:** durable jobs, category inference, duplicate review, and reproducible hotspot runs.
5. **Final verification:** performance, permissions, outage recovery, backup restoration, usability, compatibility, and documented limitations.

7.2 Definition of acceptance

A requirement is **Passed** only when its stated acceptance criterion is supported by reproducible evidence on the final build. It is **Failed** when an expected outcome is violated and **Not tested** when evidence is absent. Final baseline completion requires all 28 FR and 15 NFR records to pass, or a documented scope revision accepted by the instructor. Existing prototype features and survey claims are not substitutes for final test results.

7.3 Change control and unresolved dependencies

Requirement IDs remain stable. A change to a threshold, workflow, scope, or data privacy rule updates this report's version, rationale, affected tests, and architecture. New obligations receive new IDs rather than reusing an old one. Ward boundary accuracy, staff provisioning, labelled duplicate/cluster fixtures, object storage, production certificates, and real-data evaluation are explicit dependencies; they must not be described as completed before evidence exists.

The chosen thresholds are proposed academic acceptance targets. Instructor review may refine them before implementation is evaluated. Basic public coordinates use a grid for data minimization; it is not a formal anonymity guarantee. Synthetic classification performance establishes a reproducible baseline only, not readiness for municipal deployment.

7.4 Project sources

This report is based on supplied project materials, rather than new stakeholder interviews or fabricated test measurements:

1. Team AI_TookMyJob, *DubsiBhai Survey Report*, October 2026. [report/Survey_Report.pdf](#). Source of cover identity and stakeholder priorities.
2. *System Design*, *Core API Low-Level Design*, *ML Service Design*, and *UI Design*; project architecture documentation. Source of roles, manual moderation, public projections, workflow, and API-owned background integration.
3. Repository README, API/web/ML guides, authentication implementation, `start.ps1`, and local Compose configuration. Source of prototype status and current runtime choices; older design-document status statements may be stale.

Project repository: <https://github.com/hasanmehediii/CSE-4113-Project>